

TAD e Struct em C#

Versão redesenhada da Aula 5: mais curta, visual e pronta para prints no site.

TAD

STRUCT

C#

Ideia central

Um TAD junta os dados importantes e as operações permitidas sobre esses dados.

Por que usar?

Organiza o código, reduz repetição e deixa claro o papel de cada estrutura.

No site

Use este material como resumo visual para explicar a aula em poucos prints.

Tipo Abstrato de Dados

TAD = dados + operacoes

Um TAD descreve o que a estrutura faz antes de mostrar como ela é implementada.

Dados

São as informações guardadas pela estrutura.
Exemplo: nome, CPF, data de nascimento e e-mail de um cliente.



Operacoes

São as ações que podem ser executadas.
Exemplo: cadastrar, imprimir, buscar, alterar ou remover dados.

O que é uma struct?

Em C#, struct é um tipo de valor usado para agrupar dados relacionados em um bloco.

Resumo rápido

Struct guarda dados relacionados e pode oferecer operações para usar esses dados.

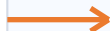
- Funciona bem para modelos pequenos e simples, como ponto, data, produto ou cliente resumido.
- Agrupa campos em uma mesma unidade, deixando o código mais organizado.
- Pode ter métodos, como `Imprimir()`, para trabalhar com os próprios dados.
- Normalmente fica na stack, uma área de memória rápida, mas menor.

TAD Cliente usando struct

A aula usa Cliente para mostrar dados + operacoes dentro de uma estrutura.

Dados sugeridos

Nome
CPF
Data de nascimento
E-mail



Operacao principal

Imprimir()

Mostra na tela os dados cadastrados do cliente de forma organizada.

Fluxo basico em C#

A sequencia mais importante: declarar, preencher e chamar uma operacao.

```
struct Cliente {  
    public string Nome;  
    public string CPF;  
    public void Imprimir() {  
        Console.WriteLine(Nome);  
        Console.WriteLine(CPF);  
    }  
}  
  
Cliente c;  
c.Nome = "Ana";  
c.CPF = "123";  
c.Imprimir();
```

Leitura do exemplo

A variavel c e do tipo Cliente. Depois de receber valores, ela usa o metodo Imprimir() para exibir seus proprios dados.

TAD ajuda a pensar antes de programar

Antes de escrever código, defina quais dados existem e quais operações fazem sentido.

1. Identifique

Quais informações a estrutura precisa guardar?

2. Modele

Quais operações devem manipular esses dados?

3. Implemente

Transforme o modelo em struct, classe ou outra estrutura.